NAME:S.DEVI SHREE

REGISTER NO:192511149

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and

hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

Code of Conduct:

I S.DEVI SHREE (192511149) certify that this submission is my

original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action.

1.Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Sequential File Organization vs Heap File Organization

Introduction

Both sequential and heap file organizations are methods of storing records in secondary storage, but they differ in the order in which records are maintained.

Comparison

Feature	**Sequential File**	**Heap File**
Record order	Sorted/ordered	No particular order
Insertion	Slow	Very fast
Deletion	Relatively slow	Easy after locating record
Sequential retrieval	Excellent	Good

Feature	Sequential File	Heap File
Exact search	Good if ordered/key known	Slow
Range queries	Excellent	Poor
Binary search	Possible	Not possible directly
Overflow	May require handling	Less concern
Maintenance	Higher	Lower

Insertion

Sequential file:

Records must be inserted in the correct position.

Example:

10 20 30 40

Inserting 25 requires:

10 20 25 30 40

This may require shifting records or maintaining overflow areas.

Therefore:

Heap file:

New records can generally be placed wherever free space is available, often at the end.

Deletion

In sequential files, deleting a record may require maintaining the ordering.

In heap files, the record can simply be marked deleted or its space can be reused.

Retrieval

Sequential files are better for:

- Sequential processing
- Range queries
- Ordered reports

Heap files are better when:

- Insertions are frequent
- Records do not need to be sorted

Example

Sequential file:

StudentID: 101 → 102 → 103 → 104

Heap file:

StudentID: 103 → 101 → 104 → 102

Conclusion

Sequential files are better for ordered retrieval and range queries, while **heap files** are better for fast insertion and simple storage.

2.Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Clustered Indexing vs Non-Clustered Indexing

Definition

A **clustered index** determines the physical/logical order in which records are stored.

A **non-clustered index** is a separate index structure containing keys and pointers to records.

Comparison

Feature	**Clustered Index**	**Non-Clustered Index**
Data order	Follows index order	Separate from data order
Number per table	Usually one	Multiple possible
Range queries	Very efficient	Less efficient
Storage	Lower additional overhead	Requires additional storage
Insert/delete	May require reorganization	Index must be updated
Sequential access	Excellent	Less efficient
Example	Index on StudentID	Index on Department

Example

Suppose a student table is physically ordered by:

StudentID

A clustered index on StudentID might look like:

101 → Record 1

102 → Record 2

103 → Record 3

104 → Record 4

A non-clustered index on Department could be:

CSE → 101, 103, 108

ECE → 102, 105

MECH → 104, 107

The actual student records do not need to be physically ordered by department.

Clustered Index Advantages

- Fast range queries
- Efficient sequential access
- Fewer disk I/Os for nearby records

Limitations

- Usually only one clustered ordering can exist.
- Insertions can be more expensive.

Non-Clustered Advantages

- Multiple indexes can be created.
- Useful for different search attributes.
- Good for exact-match queries.

Limitations

- Requires additional storage.
- May require extra disk access to retrieve actual records.

Conclusion

A **clustered index** is best when range and sequential queries are common, whereas a **non-clustered index** is useful when multiple search conditions are required.

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Primary Indexing vs Secondary Indexing

Definition

A **primary index** is generally created on the primary/ordering key of a sorted file.

A **secondary index** is created on a non-ordering attribute.

Comparison

Feature	**Primary Index**	**Secondary Index**
Based on	Primary/ordering key	Non-ordering attribute
File ordering	Usually ordered by index key	File need not be ordered
Number	Usually one	Multiple possible
Storage	Lower, especially if sparse	Usually higher
Search	Very efficient	Efficient
Duplicate values	Usually no	Common
Example	StudentID	Department

Primary Index

Example:

StudentID → Address

101 → Block 0

201 → Block 1

301 → Block 2

It can be sparse because one index entry can represent a block.

Secondary Index

Example:

Department → Pointer

CSE → Records 10, 25, 40

ECE → Records 11, 30

It generally requires more entries because multiple records may have the same value.

Storage

Primary index:

One entry per block

Secondary index:

Often one entry per record/value or one entry per distinct value

Therefore:

in many practical cases.

Search Performance

Both can provide fast searching, but primary indexing benefits from the underlying sorted order.

Secondary indexes may require an additional pointer lookup to reach the actual records.

Conclusion

Primary indexing generally requires less storage and is highly efficient for searches on the ordering key. **Secondary indexing** requires more storage but allows fast searching using additional attributes.

4. Analyze the differences between static hashing and dynamic hashing in database systems.

Static Hashing vs Dynamic Hashing

Definition

Hashing maps a search key to a bucket using a hash function.

Static Hashing

The number of buckets is fixed.

Example:

h(K) = K % 5

There are always 5 primary buckets.

0

1

2

3

4

If records increase, overflow buckets may be needed.

Dynamic Hashing

The number of buckets can grow or shrink according to the number of records.

Examples:

- Extendible hashing

- Linear hashing

Comparison

Feature	Static Hashing	Dynamic Hashing
Bucket count	Fixed	Changes dynamically
Growth	Poor	Excellent
Overflow	More likely	Controlled
Performance as DB grows	Degrades	Remains relatively stable
Implementation	Simple	More complex
Storage	Predictable	Flexible
Scalability	Limited	High

Static Hashing Problem

Suppose:

Initially: 1000 records

Buckets: 100

If records increase to:

100,000

many buckets may have long overflow chains.

This increases search time.

Dynamic Hashing

When overflow occurs:

Overflow

↓

Split bucket

↓

Redistribute records

In extendible hashing, the directory can also double.

Conclusion

Static hashing is suitable when database size is relatively stable. **Dynamic hashing** is better for growing databases because it reduces overflow and maintains efficient access.

5. Compare B-Tree indexing and B+ Tree indexing for large- scale database applications.

B-Tree vs B+ Tree

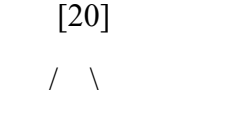
Definition

Both B-Tree and B+ Tree are balanced multiway search trees used for database indexing.

Comparison

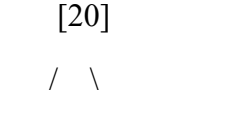
Feature	**B-Tree**	**B+ Tree**
Data in internal nodes	Yes	No
Data in leaves	Yes	Yes
Leaf nodes linked	Usually no	Yes
Fan-out	Lower	Higher
Height	Usually slightly higher	Usually lower
Range queries	Good	Excellent
Sequential access	Good	Excellent
Database usage	Less common	Very common

B-Tree



Records can exist in both internal and leaf nodes.

B+ Tree



Actual records are stored in leaf nodes.

Leaves are linked:

[10,15] → [20,30] → [40,50]

Search

Both provide approximately:

search complexity.

Range Queries

Suppose we want:

$20 \leq key \leq 50$

B+ Tree is better because after reaching the first leaf, we can follow linked leaves.

Large-Scale Databases

B+ Trees have higher fan-out because internal nodes contain only keys and child pointers. Therefore, fewer levels may be required.

Conclusion

For large-scale database applications, **B+ Tree is generally preferred** because of high fan-out, efficient range queries, sequential access and efficient disk utilization.

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Linear Hashing vs Extendible Hashing

Definition

Both are dynamic hashing techniques designed to handle database growth without using large overflow chains.

Linear Hashing

Buckets are split gradually, usually according to a split pointer.

Bucket 0

Bucket 1

Bucket 2

Bucket 3

↓

split gradually

It does not require a directory.

Extendible Hashing

Uses a directory containing pointers to buckets.

Directory

00 → Bucket A

01 → Bucket B

10 → Bucket C

11 → Bucket D

The directory can double when required.

Comparison

Feature	**Linear Hashing**	**Extendible Hashing**
Directory	No	Yes
Bucket splitting	Gradual	Triggered by overflow
Directory doubling	No	Yes
Memory overhead	Lower	Higher
Growth	Smooth	Fast/dynamic
Implementation	Moderate	More complex
Scalability	Good	Excellent

Linear Hashing

Uses a split pointer:

Split pointer → Bucket 2

Buckets are split progressively.

Extendible Hashing

When overflow occurs:

Global depth = 2

00 01 10 11

↓ overflow

Directory doubles

000 001 010 011 100 101 110 111

Conclusion

Linear hashing is useful when smooth incremental growth is desired without directory overhead. **Extendible hashing** provides fast access and flexible bucket management but requires directory storage.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Dense Indexing vs Sparse Indexing

Definition

A **dense index** contains an index entry for every record.

A **sparse index** contains index entries for only some records, usually one entry per block.

Dense Index

Key → Record

101 → R1

102 → R2

103 → R3

104 → R4

Sparse Index

Key → Block

101 → Block 1

105 → Block 2

109 → Block 3

Comparison

Feature	**Dense Index**	**Sparse Index**
Entries	Every record	Selected records
Memory	High	Low
Search	Faster	Slightly slower
File requirement	Can be flexible	Usually sorted
Maintenance	Higher	Lower

Feature	Dense Index	Sparse Index
Storage overhead	High	Low

Memory

If there are 1 million records:

Dense index may require approximately:

1,000,000 index entries

A sparse index with one entry per block might require only:

10,000 entries

depending on block capacity.

Search

Dense:

Key → Direct record pointer

Sparse:

Key

↓

Find nearest index entry

↓

Locate block

↓

Search within block

Therefore dense indexing is generally faster.

Conclusion

Dense indexes provide faster access but consume more memory, while **sparse indexes save storage but require additional searching within a block**.

8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Indexed Sequential File vs Direct File Organization

Indexed Sequential File Organization

Records are stored sequentially according to a key, and an index is maintained.

Index

↓

Sequential File

101 → 102 → 103 → 104 → 105

Direct File Organization

A hash function directly determines the location of a record.

Key

↓

Hash Function

↓

Bucket

Comparison

Feature	**Indexed Sequential**	**Direct**
Organization	Sorted/sequential	Hash-based
Exact search	Fast	Very fast
Range query	Excellent	Poor
Insert	Moderate	Fast average
Delete	Moderate	Fast average

Advantages of Indexed Sequential

- Supports sequential processing.
- Supports range queries.
- Supports indexed random access.
- Suitable for reports and ordered processing.

Limitations

- Index maintenance required.
- Insertions may cause overflow.
- More complex than heap files.

Advantages of Direct Organization

- Very fast exact-match access.
- No need to scan records.
- Average access can be:

Limitations

- Poor for range queries.
- Collision handling is required.
- Performance depends on hash function and load factor.

Conclusion

Use **indexed sequential organization** when both sequential and range access are important.
Use **direct organization** when the dominant requirement is fast exact-match retrieval.

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Single-Level Indexing vs Multi-Level Indexing

Single-Level Index

A single index directly points to the data records/blocks.

Index

|

+----→ Data

If the index becomes very large, searching the index itself may require many disk accesses.

Multi-Level Index

A second index is created over the first index.

Level 2 Index

↓

Level 1 Index

↓

Data File

Comparison

Feature	Single-Level	Multi-Level
Number of levels	One	Multiple
Index size	Can become large	Divided into smaller levels
Search	Faster for small indexes	Faster for large indexes
Disk I/O	More for large index	Fewer

Example

Suppose:

Data blocks = 1,000,000

A single huge index may itself occupy many blocks.

A multi-level index reduces the number of blocks searched:

Level 2

↓

Level 1

↓

Data

Complexity

Multi-level indexing is essentially the basic idea behind tree-based indexes.

For a B+ Tree:

Conclusion

For small databases, single-level indexing is sufficient. For very large databases, **multi-level indexing is better because it reduces disk I/O and provides scalable retrieval**.

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Hashing vs Indexing — Exact-Match and Range Queries

1. Exact-Match Query

Example:

SELECT *

FROM Student

WHERE StudentID = 105;

Hashing

Hashing is excellent for exact-match queries.

It directly identifies the bucket.

Average complexity:

Indexing

A B+ Tree can also perform exact search:

Therefore, hashing is generally faster for pure equality searches.

Range Query

Example:

SELECT *

FROM Student

WHERE CGPA BETWEEN 8.0 AND 9.0;

Hashing

Hashing is poor for range queries because nearby values can be stored in completely different buckets.

For example:

8.0 → Bucket 4

8.1 → Bucket 9

8.2 → Bucket 2

8.3 → Bucket 7

There is no natural ordering.

Therefore, range queries may require examining many buckets.

B+ Tree Index

B+ Tree maintains keys in sorted order:

7.5 → 8.0 → 8.1 → 8.2 → 8.3 → 9.0

Once the first matching key is found, linked leaves can be scanned.

Complexity:

where k is the number of records returned.

Overall Comparison

Query Type	**Hashing**	**B+ Tree Index**
Exact match	**Excellent O(1)**	Good O(log N)
Sequential access	Poor	Excellent
Insert	O(1) average	O(log N)
Delete	O(1) average	O(log N)
Equality search	**Best**	Good
Range search	Poor	**Best**

Example

For:

StudentID = 10025

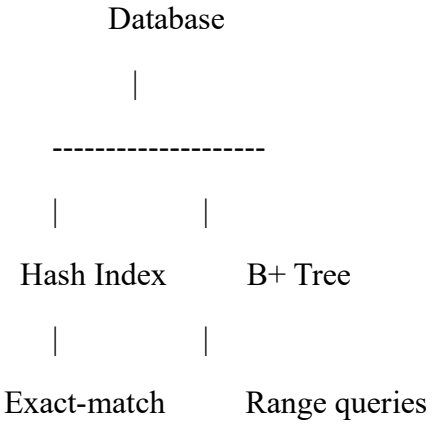
Hashing is preferred.

For:

CGPA BETWEEN 8.0 AND 9.0

B+ Tree is preferred.

Final Recommendation



Hashing is best for exact-match queries, while B+ Tree indexing is best for range queries.